

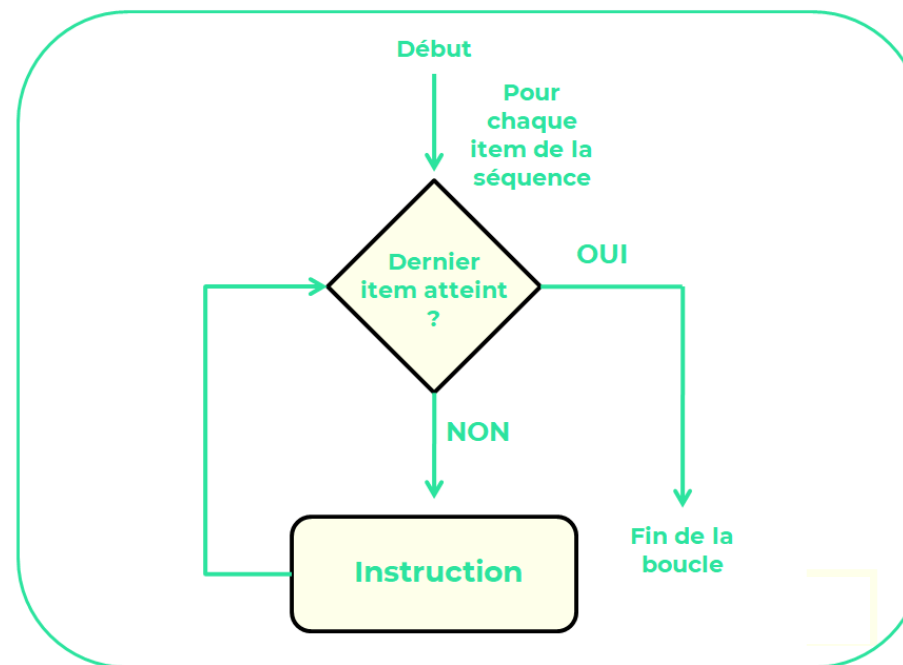
Chapitre 3 : Les structures itératives

Introduction

- On peut exécuter une action ou un ensemble d'actions non pas infiniment mais un certain nombre de fois : c'est la notion de boucles
- Il existe deux catégories d'itérations :
 - Les boucles déterministes (est une boucle dont on connaît à l'avance le nombre d'itérations)
 - Les boucles non déterministes (est une boucle dont on ne connaît pas à l'avance le nombre d'itérations)

Itérations déterministes

- Il existe une **seule** instruction permettant de faire des boucles déterministes c'est l'instruction pour
- Dans ce cas la variable prend successivement les valeurs comprises entre valeur de début et valeur de fin



Itérations déterministes

La structure « Pour faire.....Finpour »

```
Pour variable ← valeur_debut à valeur_fin Faire  
    Traitement  
FinPour
```

- `variable` : variable compteur (souvent un entier).
- `valeur_debut` : valeur initiale du compteur.
- `valeur_fin` : valeur finale jusqu'à laquelle la boucle s'exécute.
- `Traitement` : ensemble d'instructions exécutées à chaque itération.
- La variable prend successivement toutes les valeurs de `valeur_debut` à `valeur_fin` (inclus).
- Si `valeur_debut > valeur_fin`, la boucle ne s'exécute pas (zéro fois)

Itérations déterministes

Exemple

```
Algorithme AffichePour  
  
Variables :  
    i : entier  
  
Début  
    Pour i ← 1 à 5 Faire  
        Écrire(i)  
    FinPour  
Fin
```

Itérations déterministes

Incrémenter pas de 2:

```
Pour i de 1 à 10 [pas = 2] faire  
    Écrire(i)  
FinPour
```

Le compteur est à la baisse, en utilisant un pas négatif

```
Pour i de 5 à 1 [pas = -1] faire  
    Écrire(i * 10)  
FinPour
```

Itérations indéterministes

Il existe deux instructions permettant de faire des boucles indéterministes :

1. L'instruction répéter jusqu'à ce que :

```
repeter  
  instructions  
jusqu'a ce que expression booléenne
```

Qui signifie que les instructions sont exécutées jusqu'à ce que l'expression booléenne soit vraie

- Condition : condition d'arrêt et de sortie de la boucle
- Traitement : action ou ensemble d'actions à exécuter tant que la condition n'est pas vérifiée, dès qu'elle soit vérifiée, l'exécution du traitement s'arrête
- Le nombre de répétition n'est pas connu à l'avance
- Le traitement est exécuté **au moins une fois** quelque soit le résultat de la condition
- La condition **doit être initialisée** avant le début de la boucle et **doit être modifiée** à l'intérieur de la boucle

Itérations indéterministes

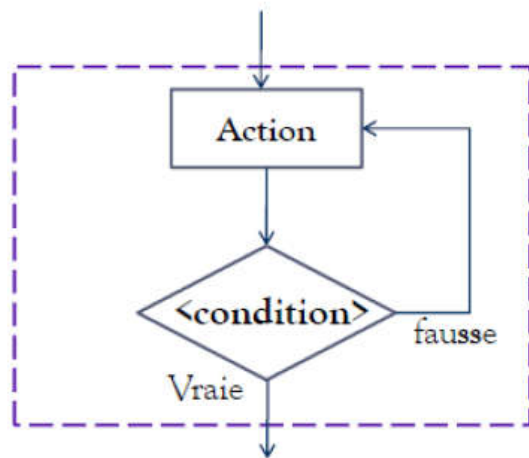
2. L'instruction tant que :

```
tant que expression booléenne faire  
    instructions  
fintantque
```

Qui signifie que tant que l'expression booléenne est vraie on exécute les instructions

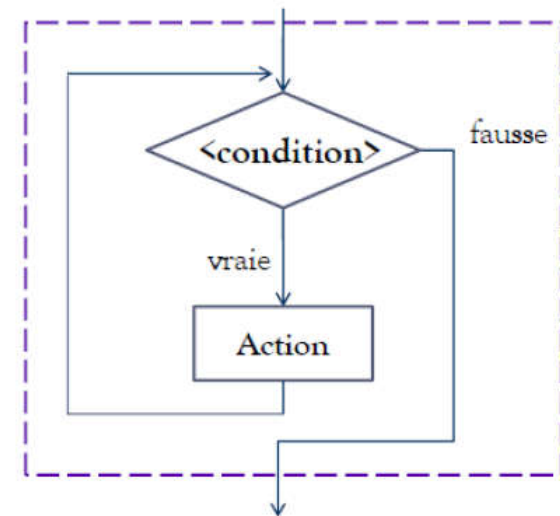
- Condition : condition de maintien de la boucle.
- Traitement : Action ou ensemble d'actions à exécuter tant que la condition est vérifiée.
- Le traitement est exécuté tant que la condition est vérifiée sinon on sort de la boucle.
- Si la **condition n'est pas vraie** dès la première exécution, la boucle **ne sera jamais exécutée** (0 fois)

Itérations indéterministes



répéter
 Action
jusqu'à (condition)

- Les instructions sont répétées jusqu'à la condition soit vérifiée > Action exécutée au moins une fois



tant que (condition) faire
 Action
fin_tant_que

- Tant que la condition est vraie, les instructions sont répétées > action exécutée 0 ou plus

Itérations indéterministes

Structure Répéter

Répéter

Instructions

Jusqu'à *Booléen*

- Arrivée à la première ligne
Répéter, **quelque soit** la valeur du **booléen**, le programme rentre dans la **boucle**.
- La machine exécute **au moins une fois** la série d'instructions de la **boucle**.
- La machine effectue **au moins une itération** dans la boucle.

Structure Tant que

Tant que *Booléen* Faire

Instructions

Fin Tant que

- Arrivée à la première ligne
Tant que, pour rentrer dans la boucle, la valeur du **booléen** doit être **VRAI**.
- La machine peut **ne jamais exécuter** la série d'instructions de la **boucle**.
- La machine peut **ne jamais effectuer d'itération** dans la boucle.

Itérations indéterministes

- Exemple

Algorithme Affiche_Repeter

Variables :
 i : entier

Début

 i ← 1

 Répéter

 Écrire(i)

 i ← i + 1

 Jusqu'à (i > 5)

Fin

Algorithme TantQue_1a5

Variables :
 i : entier

Début

 i ← 1

 Tant que i ≤ 5 Faire

 Écrire(i)

 i ← i + 1

 FinTantQue

Fin

Exercice 1

Écrire un algorithme qui demande un nombre compris entre 10 et 20, jusqu'à ce que la réponse convienne. En cas de réponse supérieure à 20, on fera apparaître un message : "Entrez un nombre plus petit !", et inversement, "Entrez un nombre plus grand !" si le nombre est inférieur à 10.

Exercice 1 - correction

Algorithme NombreEntre10et20

Variables :

n : entier

Début

Répéter

 Écrire("Entrez un nombre entre 10 et 20 : ")

 Lire(n)

 Si n > 20 alors

 Écrire("Entrez un nombre plus petit !")

 FinSi

 Si n < 10 alors

 Écrire("Entrez un nombre plus grand !")

 FinSi

Jusqu'à (n >= 10 et n <= 20)

 Écrire("Merci, le nombre est correct : ", n)

Fin

Exercice 2

Ecrire un algorithme qui demande un nombre de départ, et qui calcule la somme des entiers jusqu'à ce nombre, en utilisant les 3 types de boucles :

- Pour
- Tant que
- Répéter ... Jusqu'à

Par exemple, si l'on entre 5, le programme doit calculer :

$$1 + 2 + 3 + 4 + 5 = 15$$

Exercice 2 - correction

1. Avec la boucle Pour

Algorithme SommePour

Variables :

n, i, somme : entier

Début

Écrire("Entrez un nombre entier positif : ")

Lire(n)

somme ← 0

Pour i ← 1 à n Faire

 somme ← somme + i

FinPour

Écrire("La somme des entiers de 1 à ", n, " est : ", somme)

Fin

Exercice 2 - correction

1. Avec la boucle Tant que

Algorithme SommeTantQue

Variables :

n, i, somme : entier

Début

Écrire("Entrez un nombre entier positif : ")

Lire(n)

somme ← 0

i ← 1

Tant que i ≤ n Faire

 somme ← somme + i

 i ← i + 1

FinTantQue

Écrire("La somme des entiers de 1 à ", n, " est : ", somme)

Fin

Exercice 2 - correction

1. Avec la boucle Répéter ... Jusqu'à

```
Algorithme SommeRepeter
Variables :
    n, i, somme : entier

Début
    Écrire("Entrez un nombre entier positif : ")
    Lire(n)

    somme ← 0
    i ← 1

    Répéter
        somme ← somme + i
        i ← i + 1
    Jusqu'à (i > n)

    Écrire("La somme des entiers de 1 à ", n, " est : ", somme)

Fin
```